

ОТЧЕТ
о проектной работе
по теме: Браузерное расширение для адаптации сложности текста через AI
по дисциплине: Проектный практикум 1А

Команда: лиса милая краса

Тимлид: Гуров Михаил Георгиевич РИ-151003

Аналитик: Кучмуратов Рамазон Асомиддинович РИ-150945

Дизайнер: Юшков Артём Андреевич РИ-150946

Фронтенд-разработчик: Гуров Михаил Георгиевич РИ-151003

Бэкенд-разработчик: Гуров Михаил Георгиевич РИ-151003

1. СОДЕРЖАНИЕ

1. СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	3
2. Целевая аудитория	7
3. Определение проблемы	10
4. Подходы к решению проблемы	13
5. Анализ аналогов	16
6. Календарный план проекта	20
7. Сценарии использования	22
8. Требования к продукту и к MVP	25
9. Стек для разработки	28
10. Прототипирование	30
11. Проектирование и разработка системы	32
12. Образовательный результат	35
13. Использование генеративного ИИ в проекте	37
ЗАКЛЮЧЕНИЕ	39
СПИСОК ЛИТЕРАТУРЫ	43
ПРИЛОЖЕНИЕ А	45
ПРИЛОЖЕНИЕ Б	46

ВВЕДЕНИЕ

Актуальность темы

В условиях современного информационного общества объемы цифрового контента неуклонно растут, из-за чего возрастает потребность в инструментах быстрой и эффективной обработки текстов. Студенты младших курсов и школьники старших классов регулярно сталкиваются с необходимостью изучения сложных учебных, научных и научно-популярных материалов на русском языке. На данном этапе обучения они еще не обладают полным набором профессиональных терминов, что существенно затрудняет восприятие информации. Проведенное исследование целевой аудитории выявило конкретные барьеры при чтении: 29,2% опрошенных испытывают трудности из-за обилия незнакомых терминов, а 35,4% — из-за длинных и запутанных предложений. При возникновении таких сложностей 29,2% респондентов вынуждены искать объяснения в сторонних источниках, а 22,9% тратят время на повторное перечитывание.

На рынке существует ряд программных решений, однако они имеют системные недостатки и не адаптированы под комплексные задачи пользователей. Главная проблема большинства аналогов заключается в отсутствии автоматической работы непосредственно в браузере на лету — они требуют от пользователя ручного копирования информации в сторонние интерфейсы, что разрывает рабочий процесс. Кроме того, существующие зарубежные продукты часто не поддерживают русский язык, что делает их неприменимыми для отечественной аудитории.

Дополнительным барьером является специфика работы самих языковых моделей. Тестирование искусственного интеллекта на различных типах текстов показало, что примитивное упрощение часто приводит к критическим ошибкам: в технических и медицинских текстах термины заменяются на бытовые слова (например, точный термин заменяется на

«смешивание»), из-за чего теряется точность. В логических, философских и правовых рассуждениях нарушаются причинно-следственные связи, в инструкциях разрушается порядок действий, а в новостных материалах искажаются цифры и факты. Вследствие этого уровень полного доверия к технологиям искусственного интеллекта среди студентов составляет всего 30,4%, в то время как 28,3% респондентов открыто выражают недоверие, а 41,3% относятся к ним нейтрально.

Таким образом, формируется явное практическое противоречие: с одной стороны, у учащихся есть высокая потребность в автоматизированном упрощении сложных текстов (58,3% запрашивают замену сложных слов, а 54,2% — разбивку длинных предложений). С другой стороны, существующие инструменты требуют ручного ввода, не контролируют потерю смысла, не подсвечивают измененные фрагменты и не адаптированы под уровень конкретного пользователя. Необходимость разрешения этих проблем и создания надежного контрольного слоя для алгоритмов обработки данных определяет актуальность данной работы.

Цель и задачи проекта

Цель проекта — разработка и тестирование браузерного расширения для автоматической адаптации и изменения уровня сложности русскоязычного текстового контента страниц в режиме реального времени с использованием технологий искусственного интеллекта и механизмов сохранения точности информации.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить существующие теоретические подходы к изменению сложности текстов, проанализировать ограничения существующих программных аналогов и выявить их системные недостатки.

2. Описать функциональные и технические требования к браузерному расширению на основе запросов целевой аудитории и результатов проведенного анкетирования.
3. Разработать архитектуру и программный комплекс расширения, функционирующий без необходимости ручного копирования текста пользователем.
4. Выявить оптимальные алгоритмы интеграции искусственного интеллекта, обеспечивающие сохранение ключевых терминов с добавлением всплывающих пояснений при наведении, а также визуальное выделение измененных фрагментов текста для повышения прозрачности работы.
5. Протестировать разработанное программное решение на различных типах текстов (включая терминологические и логические материалы) для проверки корректности упрощения и стабильности интерфейса.

2. ЦЕЛЕВАЯ АУДИТОРИЯ

Тип продукта

Программный продукт представляет собой браузерное расширение для автоматической адаптации и изменения уровня сложности русскоязычного текстового контента страниц в режиме реального времени с использованием искусственного интеллекта. Продукт решает задачи замены сложных слов на простые, разбивки длинных и запутанных предложений, а также сохранения точности терминов с помощью всплывающих подсказок при наведении без необходимости ручного копирования текста в сторонние сервисы.

Тип потребителя

Основными потребителями являются студенты младших курсов (1–3 курсы) высших учебных заведений различных направлений подготовки. Данная аудитория регулярно сталкивается с учебными, научными и научно-популярными текстами, но еще не обладает полным запасом профессиональной терминологии. Также в качестве потребителей выступают школьники старших классов (9–11 классы), осваивающие профильные материалы.

Мотивация и потребность

Потребность аудитории обусловлена наличием конкретных барьеров при чтении сложных текстов:

- Необходимость преодолеть трудности, связанные с обилием незнакомых терминов, что актуально для 29,2% респондентов.
- Трудности при восприятии длинных и запутанных предложений, с которыми сталкиваются 35,4% опрошенных.

- Желание сократить время на чтение, так как в текущей ситуации 22,9% пользователей вынуждены перечитывать текст несколько раз , а 29,2% — искать объяснения в других источниках.
- Потребность в получении адаптированного текста непосредственно на веб-странице с сохранением точного смысла и прозрачности изменений , что связано с невысоким уровнем доверия к стандартным AI-инструментам (лишь 30,4% полностью доверяют искусственному интеллекту).

Ситуация, в которой приобретается продукт

Потребность в использовании расширения возникает в следующих ситуациях:

- В процессе самостоятельной подготовки к учебным занятиям, семинарам или экзаменам при работе с электронными учебниками и статьями.
- При поиске и изучении информации для написания курсовых работ, рефератов или научно-исследовательских проектов.
- Во время повседневного веб-серфинга, когда пользователь сталкивается с профильным, техническим или специфическим контентом, сложность которого мешает быстрому восприятию сути.

Место принятия решения и точка контакта

Поскольку продукт ориентирован на автоматическую работу в браузере, основными точками контакта и взаимодействия являются:

- Официальные магазины расширений, где пользователь ищет инструменты для оптимизации работы с текстом или повышения продуктивности.

- Внутренний интерфейс браузера непосредственно в момент чтения материала на любой веб-странице, где пользователю доступен функционал расширения без перехода на сторонние сайты.
- Студенческие сообщества, образовательные платформы и форумы, где целевая аудитория обменивается полезными инструментами для учебы.

3. ОПРЕДЕЛЕНИЕ ПРОБЛЕМЫ

Для выявления потребностей, барьеров и болей целевой аудитории в рамках предпроектного этапа был использован комплекс методов, позволяющий обойтись без привлечения значительных финансовых вложений. На основе специфики разрабатываемого продукта приоритетными инструментами исследования стали опрос потенциальных пользователей и детальное изучение программных решений конкурентов.

Методы выявления проблем клиентов

1. Проведение опроса Данный метод позволил напрямую взаимодействовать с потенциальными потребителями для фиксации их затруднений при работе с цифровыми текстами. Онлайн-опрос был ориентирован на выявление частоты столкновения со сложными материалами, типичных действий пользователей в случае возникновения трудностей, а также их ожиданий от гипотетического инструмента автоматизации. Опрос помог собрать первичную количественную информацию о реальных барьерах аудитории.
2. Изучение конкурентов Для определения технологических пробелов и фиксации болей реальных пользователей был проведен анализ пяти существующих решений, выполняющих смежные функции (упрощение, переформулирование, суммаризация или анализ сложности). Фокус исследования был смещен на выявление ограничений, системных недостатков и нерешенных проблем данных сервисов, что позволило определить зоны неудовлетворенного спроса.

Категории проблем клиентов

Проблемы, выявленные у целевой аудитории в процессе чтения и обработки информации, распределяются по следующим категориям:

- Отсутствие или дефицит ресурсов У пользователей фиксируется острая нехватка временных ресурсов на освоение материалов. При столкновении со сложными синтаксическими конструкциями или терминами 22,9% учащихся вынуждены повторно перечитывать текст, а 29,2% — тратить время на поиск разъяснений в сторонних источниках. Кроме того, у аудитории младших курсов и старших классов отсутствуют необходимые навыки и знания (умения) для свободного понимания узкоспециализированной профессиональной лексики.
- Неудачные попытки самостоятельного решения проблемы Попытки использовать существующие инструменты искусственного интеллекта или сторонние сервисы для упрощения текстов часто приводят к неудовлетворительному результату. Примитивное автоматическое переформулирование искажает смысл, подменяет точные термины бытовыми словами, нарушает логические связи или удаляет важные факты. Вследствие этого попытки делегировать задачу стандартным AI-моделям оборачиваются потерей точности информации, из-за чего 28,3% пользователей открыто заявляют о недоверии к текущим технологиям.

Уровни «боли» целевой аудитории

Выявленные барьеры пользователей классифицируются по трем уровням:

1. Технический уровень. Связан с непосредственными бытовыми и интерфейсными неудобствами при работе с информацией. Главная техническая боль — отсутствие автоматической работы на лету в браузере. Существующие аналоги требуют от пользователя ручного копирования фрагментов и переключения между вкладками, что разрывает рабочий процесс. Сюда же относятся технические барьеры в виде обилия запутанных предложений (35,4%) и незнакомых терминов (29,2%).

2. Влияние на финансы / Результативность. В контексте учебной деятельности данный уровень выражается в снижении эффективности работы с информацией и потере полезного времени. Из-за того, что существующие сервисы сжимают (суммаризируют) текст вместо его упрощения или полностью искажают логику рассуждений, методы решения проблемы себя не оправдывают. Пользователь теряет выгоду, получая некорректно обработанный материал, где перепутаны причинно-следственные связи или удалены ключевые цифры.

3. Личный интерес. Определяется личной мотивацией учащегося быстро, безболезненно и глубоко освоить учебный материал для успешного написания работ или подготовки к занятиям. Устранение проблемы дает прямую экономию личного времени и снижает уровень академического стресса при столкновении с тяжелыми для восприятия научными текстами. Пользователю важна прозрачность изменений и сохранение точного смысла, чтобы чувствовать уверенность в изученном материале.

4. ПОДХОДЫ К РЕШЕНИЮ ПРОБЛЕМЫ

Текущее состояние процесса

Для обоснования выбора архитектуры необходимо зафиксировать, как устроен процесс взаимодействия пользователя со сложными учебными материалами в данный момент, без использования разрабатываемой системы.

Когда студент или школьник сталкивается со сложным синтаксисом или терминами на веб-странице, стандартный сценарий выглядит следующим образом:

1. Пользователь вручную выделяет и копирует непонятный фрагмент текста.
2. Открывает новую вкладку браузера, где запущен сторонний сервис на базе искусственного интеллекта.
3. Вставляет скопированный текст в поле ввода.
4. Самостоятельно формулирует текстовый запрос (промпт) с просьбой упростить материал, объяснить термины или разбить предложения.
5. Ожидает генерации, изучает результат, после чего возвращается на исходную страницу.

Если требуется проанализировать всю страницу целиком, процесс усложняется: пользователю приходится копировать текст частями из-за ограничений на объем ввода в стандартных интерфейсах, либо тратить время на очистку скопированного материала от лишних элементов сайта (баннеров, меню, рекламы). Это приводит к потере времени, частой смене контекста и необходимости ручного написания однотипных промптов.

Альтернативные варианты решения проблемы

В процессе проектирования были рассмотрены три альтернативных варианта реализации программного продукта для решения более целевой аудитории:

- Разработка специализированного веб-портала. Создание отдельного сайта, на который пользователи заходят для загрузки документов или ручной вставки ссылок на статьи для их последующей обработки алгоритмами искусственного интеллекта. Плюс — независимость от платформы и движка конкретного браузера. Минус — пользователю приходится совершать много ручных действий по переносу данных, что не устраняет проблему разрыва рабочего процесса.
- Создание чат-бота в мессенджере. Разработка бота, который принимает текстовые сообщения от пользователя и возвращает упрощенный вариант. Плюс — мобильность и быстрота развертывания. Минус — неудобство работы с длинными академическими текстами на стационарных устройствах, отсутствие интеграции с веб-страницами и ограниченные возможности для проектирования интерфейса настроек.
- Разработка браузерного расширения с выделенным рабочим пространством. Интеграция программного модуля в веб-обозреватель, который позволяет пользователю выделить фрагмент сложного контента (либо не выделять ничего, если требуется обработать всю страницу) и нажать кнопку запуска. Расширение автоматически захватывает и сохраняет выбранный текст, после чего открывает новую вкладку, где пользователь в удобном интерфейсе выбирает необходимый уровень упрощения текста и изучает адаптированный материал. Плюс — автоматизация сбора данных на любой веб-странице, исключение ручного копирования и наличие интерфейса для точной настройки параметров обработки.

Выбор оптимального инженерного решения строится на многокритериальном анализе. Для оценки рассмотренных вариантов определен набор критериев, напрямую влияющих на удобство пользователя:

1. Автоматизация захвата контента (как отдельных фрагментов, так и всей страницы целиком) без ручного копирования.
2. Наличие интерфейса для выбора параметров и глубины упрощения текста.
3. Сохранение контекста и целостности данных при передаче в обработку.
4. Удобство чтения и работы с большими объемами информации на стационарных компьютерах.
5. Скорость доступа к функционалу (минимальное количество действий со стороны пользователя).

Оценка альтернатив выполняется в несколько итераций. Каждому критерию присваивается вес в зависимости от его приоритетности для целевой аудитории. Результат анализа показывает, насколько полно каждый вариант реализует данные требования.

На основе предварительного взвешивания очков предпочтение отдается варианту браузерного расширения с переходом в рабочую вкладку, так как оно полностью исключает рутину ручного копирования информации и предоставляет пользователю отдельное удобное пространство для кастомизации текста под его уровень подготовки. При необходимости на следующих этапах анализа могут быть привлечены эксперты для уточнения весов критериев.

5. АНАЛИЗ АНАЛОГОВ

При проектировании программного обеспечения важным этапом является исследование существующих решений. Прямые конкуренты — это аналогичное программное обеспечение на аналогичном рынке, работающее с той же целевой аудиторией, что и разрабатываемый продукт. Косвенные конкуренты — это похожее программное обеспечение с другими характеристиками или абсолютно другой продукт, но направленный на удовлетворение тех же потребностей целевой аудитории.

В рамках данной работы анализ аналогов выполнялся по шести базовым шагам:

1. Определение целей анализа (выявление слабых мест существующих решений для формирования требований к новому продукту).
2. Определение конкурентов (выбор пяти сервисов, решающих смежные задачи).
3. Определение критериев анализа (оценка языковой поддержки, интеграции в интерфейс и логики обработки текста).
4. Заполнение сравнительных таблиц на основе открытых источников, интерфейсов и отзывов.
5. Сопоставление полученных данных.
6. Формулирование итоговых выводов.

Для выявления технологических пробелов были проанализированы пять программных решений, которые выполняют задачи упрощения, переформулирования или анализа сложности текста. Поскольку на отечественном рынке отсутствуют прямые аналоги, полностью совмещающие функции AI-адаптации русскоязычного текста и формат браузерного расширения «на лету», в выборку вошли преимущественно косвенные конкуренты.

В таблице 2 представлен детальный анализ выбранных решений по ключевым критериям сравнения.

Таблица 2 – Анализ конкурентов

Критерии сравнения	Rewordify	ChatGP T/ GPT-4	Hemingway App	Lex.page	Tl;dr Chrome Extension
Поддержка русского языка	Нет (только английский)	Да	Ограничено (только подсветка)	Да	Да
Работа в браузере «на лету»	Нет	Нет (требуется ручного ввода)	Нет	Да (встроен в редактор)	Да (работает на сайтах)
Автоматическое упрощение контента	Да (качественная замена слов)	Да (высокая гибкость)	Нет (только разметка синтаксиса)	Нет (ориентирован на создание текста)	Нет (сжимает текст вместо упрощения)

Наличие встроенного AI	Нет	Да	Нет	Да	Да
------------------------------	-----	----	-----	----	----

Характеристика исследованных аналогов

- Rewordify представляет собой бесплатный веб-сервис, выполняющий качественную замену сложных лексических единиц на более простые конструкции. Однако продукт полностью ограничен английским языком, не имеет интеграции в интерфейс браузера в виде расширения и не собирает аналитические данные для оптимизации работы.
- ChatGPT / GPT-4 — универсальная чат-модель на базе искусственного интеллекта, обладающая гибкостью и глубоким пониманием контекста. Ключевой недостаток заключается в отсутствии автоматизации: процесс требует от пользователя ручного копирования фрагментов, переключения вкладок и самостоятельного написания промптов. Кроме того, базовые модели без дополнительного контрольного слоя способны искажать исходный смысл текста.
- Hemingway App — специализированный редактор сложности текста, который визуальное подсвечивает длинные предложения и перегруженную лексику. Сервис не использует технологии искусственного интеллекта и не умеет автоматически упрощать или перестраивать структуру предложений, выполняя лишь функцию статического анализа.
- Lex.page — текстовый AI-редактор со встроенным в интерфейс чатом. Данный продукт ориентирован в первую очередь на генерацию и создание нового контента, а не на адаптацию и упрощение уже существующих сторонних материалов.

- Tl;dr Chrome Extension — специализированное браузерное расширение, способное функционировать непосредственно на веб-страницах. Его основным ограничением является логика обработки данных: сервис выполняет суммаризацию (сжатие) текста, из-за чего удаляются фрагменты информации, но сама сложность оставшегося синтаксиса и терминологии не снижается.

Выводы по результатам анализа

Сопоставление полученных данных позволило выявить системные проблемы, характерные для всего спектра существующих решений:

1. Отсутствие бесшовной автоматизации. Ни один сервис не обеспечивает захват выделенного текста или всей страницы с последующим переходом в выделенное рабочее пространство для кастомизации в один клик.

2. Проблема сохранения точности. Существующие AI-инструменты не контролируют потерю смысла при переформулировании. Примитивное упрощение приводит к уничтожению научной терминологии или нарушению логических связей.

3. Непрозрачность алгоритмов. У аналогов отсутствует визуальное выделение измененных фрагментов, что снижает доверие со стороны пользователей.

4. Статичность. Сервисы не предлагают адаптацию под конкретный уровень читателя и не собирают обратную связь.

Таким образом, изучение аналогов подтверждает целесообразность разработки нового браузерного расширения. Заимствуя удачные приемы конкурентов (работа на сайтах, гибкость языковых моделей), проектируемый продукт должен устранить их ключевые недостатки за счет создания интерфейса выбора уровня сложности и интеграции механизмов контроля точности информации.

6. КАЛЕНДАРНЫЙ ПЛАН ПРОЕКТА

Название проекта:

Руководитель проекта:

Таблица 1 – Календарный план

№	Название	Ответствен ный	Длительн ость (дни)	Дата начала	Временные рамки проекта			
					1 н е д	2 н е д	3 н е д	4 н е д
Анализ								
1.1	Определение проблемы	Кучмуратов Рамазон Асомиддинови ч	3	01.04.26	*			
1.2	Выявление целевой аудитории	Кучмуратов Рамазон Асомиддинови ч	3	01.04.26	*			
1.3	Конкретизация проблемы	Кучмуратов Рамазон Асомиддинови ч	3	01.04.26	*			
1.4	Подходы к решению проблемы	Кучмуратов Рамазон Асомиддинови ч	3	01.04.26	*			
1.5	Анализ аналогов	Кучмуратов Рамазон Асомиддинови ч	3	01.04.26	*			
1.6	Определение платформы и стека для продукта	Кучмуратов Рамазон Асомиддинови ч	1	01.04.26	*			
1.7	Формулирован ие требований к MVP продукта	Кучмуратов Рамазон Асомиддинови ч	1	01.04.26	*			
1.8	Определение платформы и стека для MVP	Гуров Михаил Георгиевич	1	01.04.26	*			

1.9	Формулировка цели	Гуров Михаил Георгиевич	2	01.04.26	*	*		
1.10	Формулирование требований к продукту	Гуров Михаил Георгиевич	1	01.04.26	*	*		
1.11	Определение задач	Гуров Михаил Георгиевич	3	01.04.26	*	*		
Проектирование								
2.1	Архитектура системы (компоненты, модули системы)	Гуров Михаил Георгиевич	2	01.04.26		*	*	
2.2	Разработка сценариев использования системы	Юшков Артём Андреевич	3	01.04.26		*	*	
2.3	Прототипы интерфейсов	Юшков Артём Андреевич	1	01.04.26		*	*	
2.4	Дизайн-макеты	Юшков Артём Андреевич	1	01.04.26		*	*	
2.5	Архитектура системы (компоненты, модули системы)	Гуров Михаил Георгиевич	3	01.04.26		*	*	
Разработка								
3.1	Написание кода	Гуров Михаил Георгиевич	7	01.04.26			*	
3.2	Тестирование приложения	Гуров Михаил Георгиевич	4	01.04.26			*	
Внедрение								
4.1	Оформление MVP	Гуров Михаил Георгиевич	2	01.04.26			*	*
4.2	Внедрение MVP	Гуров Михаил Георгиевич	3	01.04.26			*	*
4.3	Написание отчета	Гуров Михаил Георгиевич	3	01.04.26				*
4.4	Оформление презентации	Кучмуратов Рамазон Асомиддинович и Юшков	3	01.04.26				*

		Артём Андреевич						
	<i>Защита проекта</i>	Все		07.06 - 15.06				*

7. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

Основным внешним агентом (актором), взаимодействующим с системой, является Пользователь — студент младших курсов или учащийся старших классов, которому необходимо оптимизировать процесс чтения и понимания сложных русскоязычных материалов.

Ниже приведено структурированное описание двух базовых сценариев взаимодействия пользователя с расширением.

Пункт 1. Сценарий использования: Упрощение выделенного фрагмента текста

- Основной актер: Пользователь.
- Цель актора: Понять смысл конкретного сложного фрагмента текста (абзаца, термина, длинного предложения) на веб-странице.
- Предусловия: Пользователь открыл в браузере веб-страницу, содержащую текстовый контент.
- Основной сценарий взаимодействия:
 1. Пользователь выделяет курсором мыши фрагмент сложного текста на веб-странице.
 2. Пользователь нажимает на кнопку вызова браузерного расширения.
 3. Система автоматически захватывает и сохраняет выделенный фрагмент текста.
 4. Система открывает новую рабочую вкладку браузера с интерфейсом управления.
 5. Пользователь выбирает необходимый уровень или параметры упрощения текста в открывшейся вкладке.
 6. Пользователь подтверждает запуск обработки контента.

7. Система передает сохраненный текст в модуль обработки данных на базе искусственного интеллекта и ожидает ответ.
 8. Система выводит адаптированный упрощенный текст на экран в интерфейсе рабочей вкладки.
 9. Пользователь читает обработанный текст.
- Постусловия: Пользователь получил доступ к упрощенной версии выбранного фрагмента текста.

Пункт 2. Сценарий использования: Упрощение контента всей веб-страницы целиком

- Основной актер: Пользователь.
- Цель актора: Ознакомиться с адаптированной и упрощенной версией всей открытой веб-страницы без необходимости разбивать её на части.
- Предусловия: Пользователь находится на веб-странице со сложным текстом, не выделяя отдельные фрагменты.
- Основной сценарий взаимодействия:
 1. Пользователь нажимает на кнопку упрощения всей страницы на панели расширения.
 2. Система автоматически считывает и сохраняет весь текстовый массив текущей веб-страницы.
 3. Система открывает новую рабочую вкладку браузера с интерфейсом настроек.
 4. Пользователь выбирает параметры глубины адаптации текста под свой уровень подготовки.
 5. Пользователь инициирует процесс автоматической адаптации.
 6. Система обрабатывает текстовый массив с помощью алгоритмов искусственного интеллекта, контролируя сохранение логики и фактов.

7. Система отображает структурированный упрощенный текст всей страницы в рабочей вкладке.
 8. Пользователь читает адаптированный материал.
- Постусловия: Полный текст веб-страницы переработан и представлен пользователю в доступном для восприятия виде в отдельном рабочем пространстве.

—

8. ТРЕБОВАНИЯ К ПРОДУКТУ И К MVP

Пользовательские требования

- Пользователь должен иметь возможность отправить текст на адаптацию в один клик, без ручного копирования и переноса данных в сторонние сервисы.
- Пользователь должен иметь возможность настраивать глубину упрощения текста под свой текущий уровень подготовки.
- Пользователь должен видеть, какие именно участки текста были изменены искусственным интеллектом, чтобы контролировать точность информации.
- Пользователь должен получать простые разъяснения профессиональных терминов прямо в тексте, без потери исходного научного или технического смысла.
- Пользователь должен быть предупрежден, если исходный текст относится к категории повышенного риска (например, юридические или философские документы), где автоматическое упрощение может исказить логику.

Функциональные требования

- Модуль захвата контента: Система должна считывать выделенный пользователем фрагмент текста на веб-странице при нажатии кнопки расширения. Если текст не выделен, система должна считывать текстовый массив всей страницы.
- Модуль маршрутизации: Система должна автоматически инициализировать открытие новой рабочей вкладки в браузере и передавать туда сохраненный текст.
- Интерфейс настроек: Система должна отображать в рабочей вкладке элементы управления для выбора уровня сложности адаптации.

- Интерфейс вывода данных: * Система должна визуально выделять цветом или подчеркиванием измененные фрагменты текста.
- Система должна интегрировать интерактивные всплывающие подсказки (tooltips) для сложных терминов, отображаемые при наведении курсора мыши.
- Модуль интеграции с AI (API): Система должна отправлять текст и выбранные параметры сложности в модель искусственного интеллекта, используя специализированные промпты, предотвращающие замену точных терминов бытовыми словами.
- Модуль валидации: Система должна анализировать тип текста и выводить предупреждающее уведомление при обнаружении логических, правовых или философских материалов.

Нефункциональные требования и ограничения

- Архитектурная среда: Продукт должен функционировать как расширение для браузеров на движке Chromium.
- Производительность: Время ожидания ответа от API и генерации страницы в рабочей вкладке не должно превышать заданного лимита, в интерфейсе должен отображаться индикатор загрузки.
- Ограничения на объем данных: Максимальный объем текста, отправляемый за один запрос, должен быть ограничен до 15 000 символов, чтобы избежать перегрузки контекстного окна модели.
- Безопасность данных: Расширение не должно сохранять личные данные пользователя на сторонних серверах, обработка текста должна происходить в рамках сессии.

Рамки MVP (Minimum Viable Product)

Что входит в MVP

- Захват только выделенного текста: Механика, при которой пользователь выделяет фрагмент сложного текста и нажимает кнопку расширения.
- Открытие рабочей вкладки: Автоматический переход в отдельное пространство после клика на расширение.
- Фиксированные уровни упрощения: Наличие нескольких фиксированных кнопок (например: «Базовое упрощение» и «Подробное объяснение терминов») вместо плавного слайдера.
- Визуальная разметка текста: Цветовое выделение измененных искусственным интеллектом предложений и базовые всплывающие подсказки для ключевых слов.
- Базовый слой промптов: Настройка инструкций для AI, предотвращающих искажение фактов в простых научно-популярных или учебных статьях.

Что переносится в полную версию

- Обработка всей страницы целиком: Разработка алгоритма автоматического извлечения основного контента (минуя интерфейс сайта) при отсутствии выделения текста.
- Плавная регулировка сложности: Реализация слайдера для точной подстройки под уровень пользователя.
- Система предупреждения о рисках: Модуль автоматической классификации жанра текста, выводящий предупреждения для юридических, философских и логических материалов.
- Сбор обратной связи: Добавление кнопок оценки качества адаптации («понятно / непонятно») для последующей корректировки промптов или обучения моделей.
- Расширение списка поддерживаемых браузеров: Оптимизация под другие популярные веб-обозреватели.

9. СТЕК ДЛЯ РАЗРАБОТКИ

В финальной сборке готового продукта технологический стек включает в себя базовые веб-стандарты и несколько специализированных клиентских утилит. Интерфейс расширения и рабочая область построены на чистом JavaScript, HTML5 и CSS3 с использованием Chromium Extension API. Для обработки и корректного отображения структурированных ответов, поступающих от локального сервера Ollama через асинхронные HTTP-запросы, в проект интегрированы готовые автономные скрипты: парсер разметки `marked.js` и математический рендерер KaTeX вместе с модулем автоматического поиска выражений `auto-render.min.js`.

Выбор этих компонентов полностью закрывает потребности интерфейса, не переусложняя общую архитектуру приложения. Подключение библиотек `marked.js` и `katex.min.js` напрямую в HTML-код позволяет полностью отказаться от бандлеров, транспиляторов и систем управления пакетами. Это обеспечивает максимальную скорость запуска разработки и простоту отладки, что критически важно для эффективного распределения ресурсов внутри команды. Код выполняется браузером напрямую, сохраняя расширение легковесным и быстрым.

Включение KaTeX и `marked.js` в стек обосновано технической необходимостью, выявленной при тестировании языковых моделей на учебных материалах. Обычный текстовый вывод искажает структуру формул и определений в физических, математических или инженерных текстах. Так как мы обязали локальный искусственный интеллект выдавать упрощенный ответ строго в формате LaTeX и Markdown, встроенные библиотеки мгновенно превращают эту техническую разметку в читаемый графический вид прямо на клиентской стороне. Это решает проблему потери точности и структуры данных без написания сложных самописных парсеров на чистом JavaScript.

Все выбранные технологии автономны и работают локально, что избавляет проект от привязки к платным облачным сервисам и внешним серверам авторизации. Используемые стандарты и библиотеки обладают долгосрочным сроком поддержки и стабильной обратной совместимостью, а их подробная документация находится в открытом доступе. Это гарантирует, что созданный код сохранит работоспособность при плановых обновлениях веб-обозревателей на движке Chromium.

10. ПРОТОТИПИРОВАНИЕ

Процесс проектирования интерфейса и логики работы расширения прошел несколько стадий, от проверки базовых механизмов до финальной сборки системы. Мы использовали итеративный подход, создавая каждый следующий вариант на основе тестирования предыдущего макета.

Первый прототип разрабатывался для проверки технической возможности захвата данных из браузера. Он состоял из минимального скрипта, который считывал выделенный пользователем текст на веб-странице и передавал его во вновь открываемую рабочую вкладку. Графического интерфейса на этом этапе не было, текст выводился в пустое окно. Этот шаг подтвердил, что программный интерфейс Chromium Extension API передает текстовые массивы без потери данных между компонентами расширения.

Второй прототип был ориентирован на настройку взаимодействия с искусственным интеллектом. Мы добавили на рабочую страницу фиксированные элементы управления для выбора режимов адаптации и реализовали прямые асинхронные запросы к локальному серверу Ollama. На этом этапе обнаружилась проблема: базовая языковая модель без жестких инструкций часто искажала смысл технических и инженерных материалов, а также полностью ломала структуру формул, уравнений и математических выражений, превращая их в нечитаемый текст (см. Приложение Б).

В третьем прототипе мы изменили подход к форматированию данных и доработали инструкции для языковой модели. Было принято решение отказаться от стандартного текстового вывода. Мы скорректировали системный промпт для Ollama, заставив искусственный интеллект генерировать упрощенный ответ строго с использованием разметки LaTeX. В код рабочей вкладки был встроен модуль рендеринга, который преобразовывал синтаксис LaTeX в графический вид. Это позволило

корректно отображать математические формулы, технические определения и структурированные блоки данных (см. Приложение Б).

Переход к готовому продукту стал результатом объединения всех проверенных компонентов. В финальной версии мы реализовали алгоритм сбора контента, который считывает текстовый массив всей веб-страницы, если пользователь не выделил конкретный фрагмент перед нажатием кнопки. Система принимает от Ollama текст в формате LaTeX, преобразует его в читаемый вид в интерфейсе рабочей вкладки и позволяет изучать адаптированный материал без потери исходной научной и технической логики. Итоговый вид элементов готовой системы представлен в графических материалах (см. Приложение А).

11. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ

Структура программного комплекса

Архитектура расширения спроектирована по принципу разделения ответственности, что позволило разбить программный комплекс на несколько автономных модулей. Первым компонентом является модуль захвата контента, который взаимодействует с Chrome API, извлекает текст с активной веб-страницы и сохраняет его в локальное хранилище браузера. Модуль пользовательского интерфейса включает в себя файлы разметки и стилей, формируя структуру рабочей области, элементы управления пресетами адаптации и зону вывода готового текста (см. Приложение А). Центральным звеном выступает управляющий контроллер приложения, написанный на чистом JavaScript. Он обрабатывает события интерфейса, координирует передачу данных между модулями и управляет состояниями экрана загрузки. Блок интеграции с локальной языковой моделью отвечает за построение сетевых асинхронных запросов к серверу Ollama и извлечение сырого текстового ответа. Завершает структуру модуль постпроцессинга и рендеринга контента, в который интегрированы внешние парсеры, предназначенные для очистки полученных данных, преобразования Markdown-разметки и отрисовки математических выражений.

Алгоритм работы приложения

Логика работы приложения построена на последовательности действий, обеспечивающих обработку текста без потери структуры. На этапе инициализации контентный скрипт считывает текстовый массив со страницы и помещает его в хранилище под определенным ключом. При запуске интерфейса рабочей области контроллер извлекает этот текст и отображает его в исходном виде. Когда пользователь выбирает один из режимов адаптации в боковой панели, алгоритм перехватывает клик и извлекает

соответствующую текстовую инструкцию роли. Далее контроллер объединяет текст роли с глобальным системным правилом форматирования, запрещающим использование таблиц, и присоединяет к этой строке исходный технический текст. На экран выводится индикатор ожидания, а сформированный массив отправляется методом POST на локальный порт сервера искусственного интеллекта. После получения ответа алгоритм очищает строку от сервисных символов разметки блоков кода, выполняет четырехкратное экранирование символов разметки LaTeX для предотвращения сбоев в работе текстового парсера, трансформирует Markdown-теги в HTML-элементы и инициирует сканирование DOM-дерева модулем отображения формул. Окончательно сформированный текст подставляется в окно чата, а индикатор загрузки переводится в скрытый режим.

Схема потоков данных

Взаимодействие модулей и движение информации внутри комплекса подчинены линейной схеме потоков данных (см. Приложение А). Исходный информационный поток берет начало из объектной модели веб-страницы, откуда текст передается в локальное хранилище Chrome. Из хранилища данные считываются контроллером интерфейса. При активации пользовательского запроса поток данных разветвляется: исходный текст объединяется с конфигурационным массивом промпта и направляется по сетевому протоколу HTTP в локальную нейросеть Ollama. Обратный поток возвращает сырой неструктурированный текст в модуль контроллера. Оттуда текстовый массив поступает на вход парсера Markdown, который преобразует его в промежуточный HTML-код. На финальном этапе этот код обрабатывается математическим рендерером, который заменяет текстовые маркеры уравнений на графические векторные элементы, после чего обновленный поток данных направляется непосредственно в DOM-структуру активного окна для демонстрации пользователю.

Процесс разработки сложных блоков

При реализации программного комплекса основная сложность заключалась в разработке двух архитектурных узлов: системы разделения длинных текстов и блока бесконфликтного отображения разметки. В процессе написания скрипта обработки больших массивов данных возникла необходимость обходить ограничения контекстного окна локальных моделей. Для решения этой задачи был написан алгоритм циклического деления строки по фиксированному количеству символов, который предварительно отправляет текстовые фрагменты для извлечения сквозных тем, а затем агрегирует результаты. Вторым барьером стала интеграция парсера Markdown и математического рендерера KaTeX в рамках одного текстового контейнера. Стандартный парсер воспринимал служебные символы формул как элементы форматирования текста, что приводило к разрушению структуры уравнений. Мы решили эту проблему на уровне управляющего контроллера, внедрив регулярные выражения, которые принудительно экранируют специфические разделители LaTeX перед вызовом основного парсера и корректно передают обработанный HTML-код в модуль визуализации математики (см. Приложение Б).

Гуров Михаил — Программист и Тимлид

В этом семестре передо мной стояла двойная задача: подтянуть навыки управления проектом и разобраться с технической стороной интеграции нейросетей. Моя образовательная цель заключалась в том, чтобы получить реальный опыт руководства командой, выявить свои слабые места как тимлида и параллельно на практике освоить механику асинхронных запросов к моделям искусственного интеллекта напрямую из браузерного расширения. Критериями успешности я определил создание рабочей сборки расширения, отсутствие серьезных конфликтов в коллективе и четкое понимание своих зон роста.

Свой прогресс я оцениваю на 8 баллов из 10. Проект помог мне понять, что как организатор я действую нормально — способностей хватает, чтобы собрать ребят, определить общую цель и довести техническую часть до релиза. Но работа подсветила и мои проблемы в качестве руководителя. Я понял, что функционирую скорее как исполнитель: мне дали конкретную задачу — я ее сделал. Мне тяжело самостоятельно контролировать долгосрочные дедлайны без внешних напоминаний, из-за чего мы пару раз сдвигали сроки просто потому, что я вовремя не уточнил точные даты. Это важный опыт, и в будущем я буду жестче подходить к календарному планированию внутри команды. При этом в плане работы с ИИ и Chromium API я попрактиковался отлично, тут цель выполнена полностью.

Кучмуратов Рамазон — Аналитик

Моя образовательная цель на этот проект — получить практический опыт в системном анализе, освоить рабочие подходы к определению проблем и поиску путей их решения. Мне хотелось научиться не просто фиксировать жалобы пользователей, а четко локализовать технические барьеры и переводить их в понятные требования для разработки. Критериями достижения цели я считаю вовремя составленные аналитические записки и успешное обоснование архитектуры приложения на этапе проектирования.

Я оцениваю свой прогресс на 10 баллов из 10. Задачи решались оперативно и без лишних усложнений. Основная аналитическая проблема, с которой я столкнулся, заключалась в том, что студенты быстро теряли концентрацию и упускали суть при чтении больших объемов сложной технической литературы. Для ее решения я предложил подход с жестким ограничением контекста. Мы внедрили требования к ролевым инструкциям

для искусственного интеллекта, которые отсекают избыточную информацию и заставляют модель выдавать только структурированную суть. Проблема была локализована, требования программисту я передал вовремя, все аналитические задачи закрыты.

Юшков Артём — Дизайнер

Я шел в проект с конкретной целью — попробовать себя в проектировании реальных интерфейсов и закрепить навыки работы в графических редакторах Adobe Photoshop и GIMP. Мне хотелось выйти за рамки чистой теории и создать визуальное оформление для расширения, которым людям будет действительно удобно пользоваться в повседневной учебе. Критериями успеха для меня стали полностью готовый макет боковой панели, комплект иконок для расширения во всех нужных для Chromium разрешениях и отсутствие замечаний по визуалу со стороны команды.

Свой результат я оцениваю на 10 баллов из 12. Мне очень понравилось работать над этим проектом, процесс принес много удовольствия и реального драйва. Было безумно интересно проектировать интерфейс боковой панели, подбирать цвета и шрифты так, чтобы глаза пользователей не уставали при долгом чтении адаптированных текстов. В ходе работы я полностью разобрался с подготовкой и нарезкой графических ассетов для веба, стилизовал рабочую область и получил отличный практический опыт. Работа принесла массу положительных эмоций, а созданный интерфейс готов и полностью выполняет свою задачу.

13. ИСПОЛЬЗОВАНИЕ ГЕНЕРАТИВНОГО ИИ В ПРОЕКТЕ

В процессе создания расширения генеративные языковые модели привлекались исключительно в качестве вспомогательного инструмента. Роль искусственного интеллекта была ограничена функциями текстового ассистента и линейного редактора, в то время как вся ключевая инженерная и аналитическая работа выполнялась разработчиками самостоятельно. В качестве ИИ-агентов использовались большие языковые модели общего назначения в рамках стандартных диалоговых сессий. Инструмент применялся для ускорения рутинных операций, связанных с оформлением технической документации и точечным поиском дефектов в коде.

Контекст для моделей задавался прямым и прагматичным способом. При поиске программных ошибок в окно запроса передавался конкретный проблемный участок скрипта на чистом JavaScript вместе с описанием логического сбоя или текстовой ошибкой из консоли браузера. Для генерации текстовых материалов модели получали краткие исходные тезисы с четкой инструкцией: расширить объем описания без потери первоначального технического смысла, подобрать альтернативные формулировки фраз для интерфейса или адаптировать стиль изложения под требования отчета. Обработка результатов всегда включала обязательный ручной аудит со стороны разработчика. Сгенерированные варианты текстов проходили обязательную вычитку и сокращение, чтобы убрать речевую избыточность, а подсказки по коду анализировались на предмет логического соответствия архитектуре Chromium Extension API перед ручным переносом изменений в файлы проекта.

В соответствии с политикой прозрачности в проекте установлено четкое разделение труда между человеком и искусственным интеллектом. Разработчики полностью самостоятельно спроектировали архитектуру расширения, определили состав модулей, написали весь рабочий код на

чистом JavaScript, создали структуру разметки HTML5 и стилей CSS3, а также настроили логику интеграции и вызова сторонних утилит marked.js и KaTeX. Искусственный интеллект использовался локально и точно. Его задачи свелись к автоматизации рутинного расширения объема текста для некоторых разделов пояснительной записки, подбору синонимов для элементов управления и оперативной помощи в поиске мелких багов при отладке асинхронных сетевых запросов к локальному серверу Ollama. Подобное распределение ролей позволило сохранить полный контроль над стабильностью программного комплекса и обеспечить надежность итогового продукта.

ЗАКЛЮЧЕНИЕ

Выполнение данного проекта позволило полностью пройти весь жизненный цикл разработки программного обеспечения, начиная от предварительного анализа требований конечных пользователей и заканчивая сборкой стабильно функционирующего программного комплекса. Все задачи, которые были сформулированы на начальных этапах проектирования, решены в полном объеме, что привело к созданию работоспособного расширения для браузеров на движке Chromium, интегрированного с локальными инструментами искусственного интеллекта. Разработка велась в рамках методологии нисходящего проектирования, при которой общая система последовательно разбивалась на изолированные программные модули, что упростило процесс отладки и исключило появление критических конфликтов в коде на этапе финальной сборки.

Если рассматривать полученные результаты в строгом соответствии с хронологией поставленных во введении задач, то на первом этапе был успешно осуществлен детальный анализ предметной области и потребностей целевой аудитории. Проведенное исследование подтвердило, что студенты технических специальностей и старшие школьники регулярно сталкиваются с проблемой восприятия учебной и научной литературы из-за избыточной плотности терминологии и отсутствия оперативных инструментов для контекстного разъяснения сложных понятий. Анализ существующих аналогов показал, что большинство из них требуют от пользователя совершения лишних действий, таких как копирование текста и перенос его на сторонние веб-ресурсы, что нарушает непрерывность образовательного процесса. Создание расширения, работающего непосредственно внутри активной вкладки браузера, позволило полностью устранить этот барьер.

Вторая задача, связанная с преодолением ограничений генеративных моделей при обработке точных дисциплин, была решена в ходе

итерационного прототипирования. Нам удалось справиться с проблемой искажения исходного смысла и разрушения структуры математических выражений, которая часто возникает при стандартных запросах к языковым моделям. Путем модификации системных промптов и внедрения жестких правил форматирования мы обязали локальную модель Ollama возвращать адаптированный текст строго в формате Markdown и LaTeX. Третья задача, охватывающая непосредственное написание и сборку кода, была выполнена силами команды без привлечения тяжеловесных сторонних фреймворков. Написание управляющих скриптов на чистом JavaScript позволило создать отзывчивый интерфейс рабочей области, координирующий извлечение данных из локального хранилища браузера, управление состояниями загрузки и передачу текстовых массивов по сетевым протоколам.

Анализ готового программного продукта позволяет выделить ряд существенных положительных сторон реализованного архитектурного решения. Ключевым достоинством является автономность и конфиденциальность всей системы. Поскольку обработка текстов и генерация упрощенных версий происходят на локальном сервере Ollama, личные данные и содержание изучаемых документов не передаются на внешние сторонние серверы, что полностью исключает риски утечки информации. Отказ от использования коммерческих облачных API избавил проект от необходимости интеграции платежных шлюзов и зависимости от зарубежных провайдеров. С технической стороны выбор чистого JavaScript, HTML5 и CSS3 обеспечил высокую скорость инициализации приложения, легковесность расширения и отсутствие необходимости в сложной предварительной сборке кода. Интеграция парсера `marked.js` и математического рендерера KaTeX позволила добиться корректного и визуально понятного отображения формул, уравнений и структурированных блоков данных, что делает продукт применимым для изучения точных и инженерных наук.

С другой стороны, разработанное решение имеет определенные отрицательные особенности и ограничения, которые необходимо учитывать при его эксплуатации. Самым важным недостатком является прямая зависимость скорости генерации адаптированного текста от аппаратных ресурсов компьютера пользователя. На рабочих станциях с низкой производительностью процессора или малым объемом видеопамати время ожидания ответа от локальной модели может существенно возрасти, что снижает комфорт от использования расширения. Кроме того, ограничения контекстного окна используемой модели накладывают лимиты на объем одновременно обрабатываемого текста. Для минимизации этой проблемы в управляющий контроллер был внедрен алгоритм принудительного дробления строк на фрагменты, однако это может приводить к частичной потере сквозных логических связей в очень длинных документах. Наконец, текущая версия продукта жестко привязана к программным интерфейсам Chromium Extension API, что исключает его использование в веб-обозревателях на других движках без существенной переработки исходного кода.

На текущем этапе развития проекта все программные компоненты полностью объединены, логические ошибки в коде устранены, и сервис готовится к официальному запуску и первому этапу тестирования на реальных пользователях. Для проведения закрытого пилотного тестирования сформирована фокус-группа, состоящая из студентов технических направлений. В ходе этого этапа планируется собрать первичные эксплуатационные метрики, оценить реальную скорость обработки запросов на различных аппаратных конфигурациях, а также проверить удобство интерфейса рабочей области в условиях повседневных учебных нагрузок. Обратная связь от участников пилотной группы ляжет в основу последующих обновлений и позволит выявить скрытые дефекты, которые могли быть упущены в процессе изолированной отладки.

Перспективы дальнейшей разработки темы охватывают несколько ключевых направлений модернизации как интерфейсной, так и алгоритмической части. В планах развития продукта стоит адаптация и портирование расширения под другие популярные браузерные движки для расширения потенциальной аудитории. Для повышения удобства взаимодействия пользователей с системой планируется заменить фиксированные кнопки режимов адаптации на интерактивный слайдер, который позволит плавно регулировать глубину упрощения текста в зависимости от текущего уровня подготовки читателя. Важной задачей для следующих версий является создание модуля автоматического распознавания жанров контента. Этот блок позволит системе на лету определять, что исходный текст относится к категории повышенного риска, например, к юридическим документам или сложным философским трактатам, и выводить предупреждение о том, что автоматическое упрощение может исказить строгую логику оригинала. Также планируется реализовать локальный механизм накопления оценок пользователей для последующей автоматической корректировки системных инструкций языковой модели.

СПИСОК ЛИТЕРАТУРЫ

1. Веб-технологии для разработчиков Обзор протокола HTTP / Веб-технологии для разработчиков. – 2022. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP/Overview> (дата обращения: 30.04.2022).
2. Вигерс К. Разработка требований к программному обеспечению / К. Вигерс, Дж. Битти. – 3-е изд. – Москва: Русская редакция, 2014. – 736 с. – ISBN 978-5-9909805-3-2.
3. Кинзябулатов Р. IDEF0. Знакомство с нотацией и пример использования / Рамиль Кинзябулатов. – 2022. – URL: <https://trinion.org/blog/idef0-znakomstvo-s-notaciey-i-primer-ispolzovaniya> (дата обращения: 30.04.2022).
4. Коберн А. Современные методы описания функциональных требований к системам / А. Коберн. – Москва: Издательство «Лори», 2012. – 264 с. – ISBN 978-5-85582-326-4.
5. Мидоус Ф. Введение в протоколы HTTP и HTTPS / Филлип Мидоус. – 2022. – URL: <https://docs.microsoft.com/ru-ru/azure/rtos/netx-duo/netx-duo-web-http/chapter1> (дата обращения: 05.05.2022).
6. Best Programmer Функциональные и нефункциональные требования: полное руководство / Best Programmer. – 2021. – URL: <https://bestprogrammer.ru/izuchenie/funktsionalnye-i-nefunktsionalnye-trebovaniya-a-polnoe-rukovodstvo> (дата обращения: 15.05.2022).
7. Google Chrome Браузер от Google / Google Chrome. – 2022. – URL: <https://www.google.ru/chrome/> (дата обращения: 30.04.2022).
8. JavaScript WebSocket / JavaScript. – 2022. – URL: <https://learn.javascript.ru/websocket> (дата обращения: 02.05.2022).
9. VC.RU Анализ конкурентов на практике / VC.RU. – 2020. – URL: <https://vc.ru/marketing/181297-analiz-konkurentov-na-praktike-10-shagov-shablon> (дата обращения: 30.04.2022).

10. VC.RU Как составить портрет клиента (целевой аудитории) / VC.RU. — 2020. — URL: <https://vc.ru/marketing/156147-kak-sostavit-portret-klienta-celevoy-auditorii-instrukciya-s-primerami> (дата обращения: 30.04.2022).

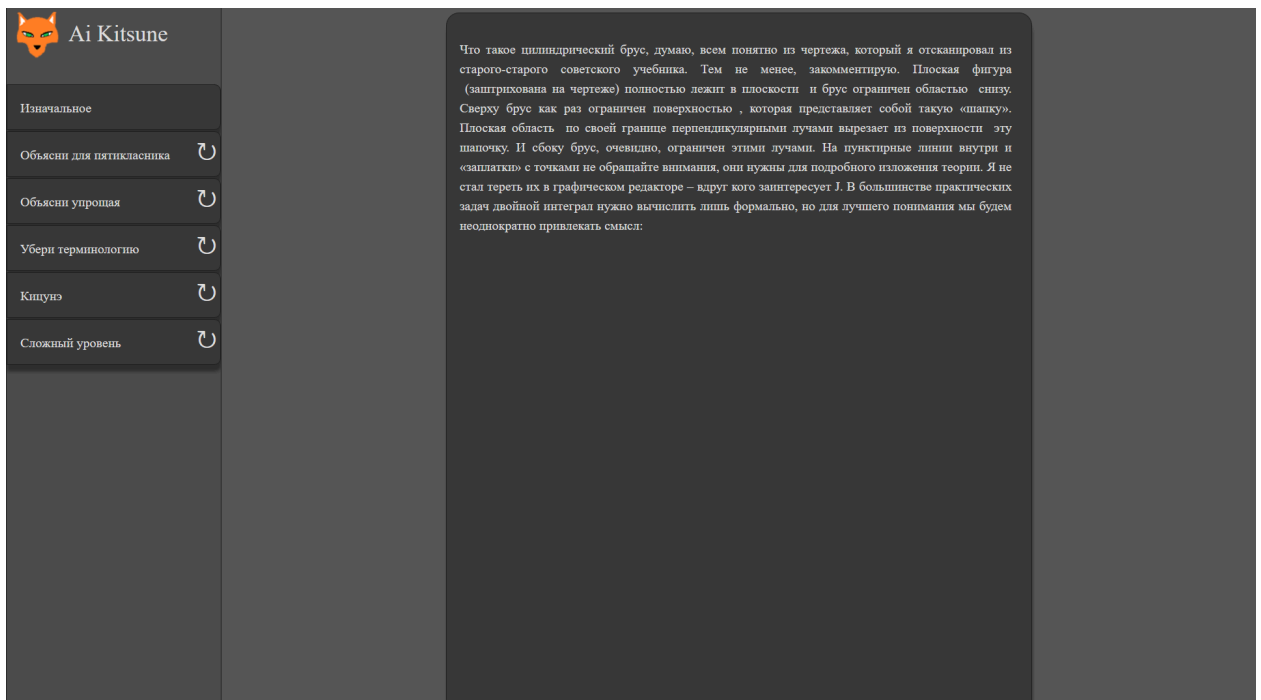


Рисунок 1. Интерфейс

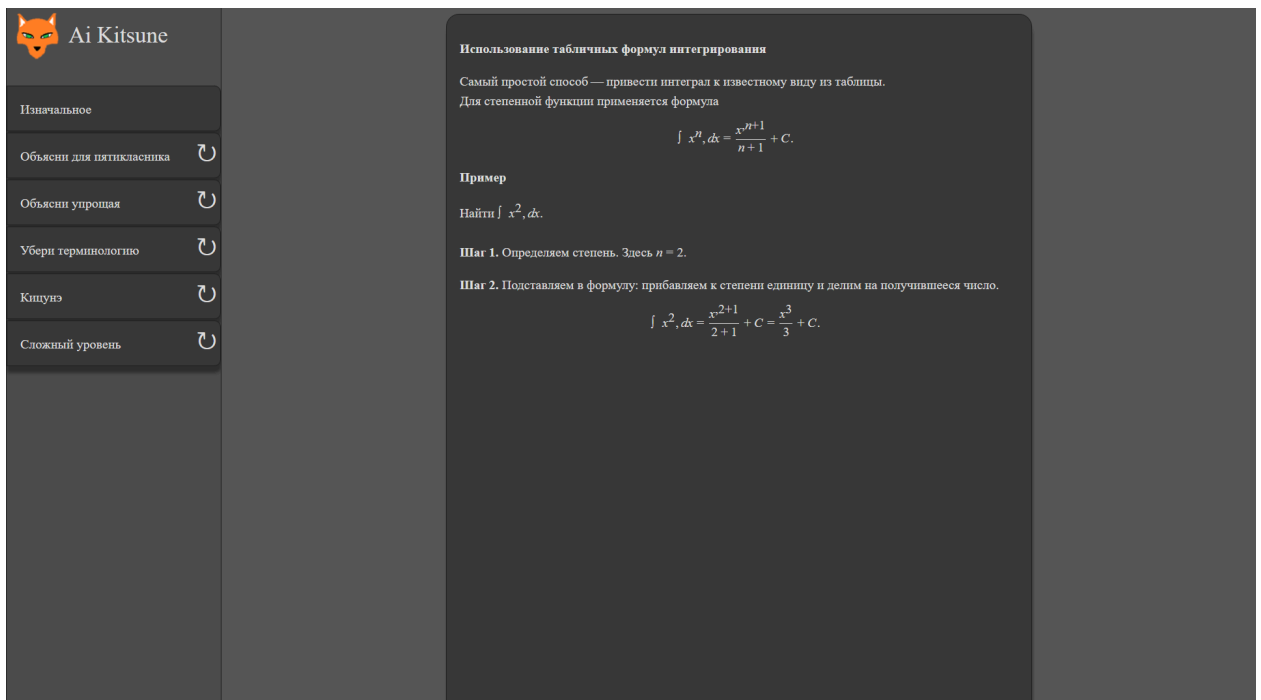


Рисунок 1. Вывод математических формул в LaTeX